

DAY 2 (afternoon): Login with Google (OAuth 2.0), FRONTEND

Angular 16 + Material, Midnight theme. **2 new/changed files to copy whole, 5 small edits.** The email and password form and every existing id stay exactly as they are.

0. What the user sees

- On `/login`: the normal form, a line "or", and Google's dark **Sign in with Google** button (registered people only; an unknown e-mail shows "No account found ... register first" with a Register link). On `/registration`: the same style button **Sign up with Google**, which stores the account and logs the person in. If `googleClientId` is empty, the whole Google block is hidden and Google's script is never requested.
- A good Google token: the same session as a normal login (token, role, session flag). A new customer lands on the customer-details page; an existing one on the dashboard; an instructor on the application page; an admin is refused with "Admin accounts must log in with their email and password."
- A person who registered with the form and has **not verified the phone** is sent to `/verify-phone` with their e-mail filled in (the e-mail is read from Google's token only to pre-fill the page; the server decides).
- A bad token: "Google sign-in failed. Please try again."

Honest status. Real browser, pretend backend, stand-in Google button: **14 checks, 0 failures** (script requested once, button drawn with dark theme and "continue with" text, bad token, admin refused, good token stores the session, empty client id hides everything, phone width) plus **3 checks** for the unverified-phone rule. The real Google pop-up was not run (it needs your Client ID). `ng test` was not re-run.

1. Edits

1. `environment.ts` and `environment.prod.ts`: add the Client ID

In `angularapp/src/environments/environment.ts` add one line inside the object (do not forget the comma after the previous line):

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080',
  aiSearchEnabled: true,
  // OAuth: the public "Client ID" from Google Cloud Console (not a secret). Leave '' to hide the Google button.
  googleClientId: '1234-abcd.apps.googleusercontent.com' // <-- paste YOUR Client ID here
};
```

Do the same in `environment.prod.ts` (add the line `googleClientId: '...'` after `aiSearchEnabled: true,`).

If you leave it as `''`, the Google button simply does not appear and nothing else changes.

2. `constant.ts` : two small edits

Edit 1. Inside `export const API = { ... }` add one line (for example right after the `UPLOAD_IMAGE` line, or after `LOGOUT`):

```
GOOGLE_LOGIN: '/api/auth/google',
```

Edit 2. Find this line near the bottom:

```
export const PUBLIC_API_PATTERN = /\api\/(login|register|refresh|logout)$/;
```

Change it to (only `|auth\/google` is added inside the brackets):

```
export const PUBLIC_API_PATTERN = /\api\/(login|register|refresh|logout|auth\/google)$/;
```

Why: `AuthInterceptor` uses this list to know which calls are login calls (it must not attach an old token to them, and must not try to refresh on a 401 from them).

3. `auth.service.ts` : add ONE method

Open `angularapp/src/app/services/auth.service.ts`. Find the comment `/** Gets a new token using the refresh cookie.` and add this method **just above it** (inside the class):

```
/** OAuth: sends the Google ID token to the server and starts the same session as a normal login. */
loginWithGoogle(credential: string): Observable<any> {
  return this.http.post<any>(`${this.apiUrl}${API.GOOGLE_LOGIN}`, { credential }, { withCredentials: true
}).pipe(
  tap((res: any) => this.applySession(res))
);
}
```

Nothing else in this file changes. It reuses your existing `applySession`, so the token, role and user id are stored exactly like a normal login.

3b. Also changed for "Sign up with Google"

`services/auth.service.ts` (`loginWithGoogle(credential, mode)` now sends `mode`) and the three **registration** files `components/registration/registration.component.ts/html/css` (they add the Google sign-up button). The complete registration files are in **DAY1-OTP-FRONTEND.pdf** (section "registration") and in the merged zip.

4. NEW file `services/google-auth.service.ts`

`angularapp/src/app/services/google-auth.service.ts` (new file)

```
import { Injectable } from '@angular/core';
import { environment } from '../../../environments/environment';

/**
 * Loads Google's "Sign in with Google" script and draws its button.
 * The button gives us a Google ID token ("credential"); LoginComponent sends it to the backend.
 * When environment.googleClientId is empty nothing is loaded and the button stays hidden.
```

```

*/
@Injectable({
  providedIn: 'root'
})
export class GoogleAuthService {
  private static readonly SCRIPT_URL = 'https://accounts.google.com/gsi/client';
  private scriptLoaded: Promise<void> | null = null;

  /** True when a Google client id is configured. */
  get enabled(): boolean {
    return !!environment.googleClientId;
  }

  private loadScript(): Promise<void> {
    if (!this.scriptLoaded) {
      this.scriptLoaded = new Promise<void>((resolve, reject) => {
        const script = document.createElement('script');
        script.src = GoogleAuthService.SCRIPT_URL;
        script.async = true;
        script.defer = true;
        script.onload = () => resolve();
        script.onerror = () => {
          this.scriptLoaded = null; // allow a retry later
          reject(new Error('Could not load Google sign-in'));
        };
        document.head.appendChild(script);
      });
    }
    return this.scriptLoaded;
  }

  /** Draws the Google button inside "host" (text: "Sign in with Google" or "Sign up with Google");
  onCredential gets the Google ID token. */
  async renderButton(host: HTMLElement, onCredential: (credential: string) => void,
    text: 'signin_with' | 'signup_with' | 'continue_with' = 'signin_with'): Promise<void>
  {
    if (!this.enabled) {
      return;
    }
    await this.loadScript();
    const google = (window as any).google;
    google.accounts.id.initialize({
      client_id: environment.googleClientId,
      callback: (response: { credential: string }) => onCredential(response.credential)
    });
    google.accounts.id.renderButton(host, {
      type: 'standard',
      theme: 'filled_black',
      size: 'large',
      shape: 'pill',
      text,
      width: Math.min(Math.max(host.clientWidth || 320, 200), 400)
    });
  }
}

```

5. `components/login/login.component.ts` : replace the whole file with THIS one

This is the Google guide's file **plus** the three later additions (the instructor redirect twice, the "verify your mobile number" redirect for form logins, and the same redirect for Google logins). Use it instead of any older copy.

`angularapp/src/app/components/login/login.component.ts` (replace the whole file)

```

import { AfterViewInit, Component, ElementRef, NgZone, ViewChild } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

```

```

import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';
import { CustomerService } from '../../services/customer.service';
import { GoogleAuthService } from '../../services/google-auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent implements AfterViewInit {
  loginForm: FormGroup;
  errorMessage = '';
  loading = false;
  showPassword = false;
  /** True when Google said this e-mail has no account: the page then offers a link to register. */
  notRegistered = false;
  /** True when a Google client id is configured, so the Google button is shown. */
  googleEnabled = false;
  @ViewChild('googleButton') googleButton?: ElementRef<HTMLElement>;

  constructor(
    private fb: FormBuilder,
    private auth: AuthService,
    private customerService: CustomerService,
    private router: Router,
    private google: GoogleAuthService,
    private zone: NgZone
  ) {
    this.googleEnabled = this.google.enabled;
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  get email() { return this.loginForm.get('email'); }
  get password() { return this.loginForm.get('password'); }

  onSubmit(): void {
    if (this.loginForm.invalid || this.loading) { return; }
    this.loading = true;
    this.errorMessage = '';
    this.auth.login(this.loginForm.value).subscribe({
      next: (res: any) => {
        if (res.role === 'INSTRUCTOR') {
          this.loading = false;
          this.router.navigate(['/instructor-register']);
          return;
        }
        if (res.role === 'ADMIN') {
          this.loading = false;
          this.router.navigate(['/dashboard']);
          return;
        }
        // customers must have filled in their details once before they can use the app
        this.customerService.viewCustomerByUserId().subscribe({
          next: () => { this.loading = false; this.router.navigate(['/dashboard']); },
          error: (err) => {
            this.loading = false;
            if (err.status === 404) {
              this.router.navigate(['/customerdashboard']);
            } else {
              this.errorMessage = 'Could not load your profile. Please try again.';
            }
          }
        });
      },
      error: (err) => {
        this.loading = false;
        if (err.status === 401) {

```

```

        this.errorMessage = 'Invalid email or password';
    } else if (err.status === 403 && String(err.error?.message || '').includes('verify your mobile
number')) {
        this.router.navigate(['/verify-phone'], { queryParams: { email: this.loginForm.value.email } });
    } else if (err.status === 429) {
        this.errorMessage = 'Too many attempts. Please wait a while and try again.';
    } else {
        this.errorMessage = 'Unable to reach the server. Please try again later.';
    }
    }
    });
}

// ----- Login with Google (OAuth 2.0) -----

ngAfterViewInit(): void {
    if (this.googleEnabled && this.googleButton) {
        this.google.renderButton(this.googleButton.nativeElement, (credential) =>
this.onGoogleCredential(credential), 'signin_with')
        .catch(() => { this.errorMessage = 'Google sign-in could not be loaded. Check your internet
connection.'; });
    }
}

/** Reads the e-mail out of Google's ID token (a JWT) only to pre-fill the verify page. The server never
trusts this. */
private emailFromGoogleCredential(credential: string): string {
    try {
        const part = credential.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
        const email = JSON.parse(atob(part)).email;
        return typeof email === 'string' ? email : '';
    } catch {
        return '';
    }
}

/** Google calls this outside Angular, so we step back into the Angular zone first. */
private onGoogleCredential(credential: string): void {
    this.zone.run(() => {
        if (this.loading) { return; }
        this.loading = true;
        this.errorMessage = '';
        this.notRegistered = false;
        this.auth.loginWithGoogle(credential, 'login').subscribe({
            next: (res: any) => this.goAfterLogin(res),
            error: (err) => {
                this.loading = false;
                if (err.status === 404) {
                    // the Google e-mail is not registered: nothing was created
                    this.notRegistered = true;
                    this.errorMessage = 'No account found for this Google e-mail. Please register first.';
                } else if (err.status === 401) {
                    this.errorMessage = 'Google sign-in failed. Please try again.';
                } else if (err.status === 403 && String(err.error?.message || '').includes('verify your mobile
number')) {
                    // registered with the form earlier and never verified the phone: finish that first
                    const email = this.emailFromGoogleCredential(credential);
                    if (email) {
                        this.router.navigate(['/verify-phone'], { queryParams: { email } });
                    } else {
                        this.errorMessage = 'Please verify your mobile number first. Log in with your email and
password to start.';
                    }
                } else if (err.status === 403) {
                    this.errorMessage = 'Admin accounts must log in with their email and password.';
                } else if (err.status === 400) {
                    this.errorMessage = 'Login with Google is not set up on the server yet.';
                } else {
                    this.errorMessage = 'Unable to reach the server. Please try again later.';
                }
            }
        })
    })
}

```

```

    });
  });
}

/** Same rule as the normal login: admins go to the dashboard, customers need their details saved once.
 */
private goAfterLogin(res: any): void {
  if (res.role === 'INSTRUCTOR') {
    this.loading = false;
    this.router.navigate(['/instructor-register']);
    return;
  }
  if (res.role === 'ADMIN') {
    this.loading = false;
    this.router.navigate(['/dashboard']);
    return;
  }
  this.customerService.viewCustomerById().subscribe({
    next: () => { this.loading = false; this.router.navigate(['/dashboard']); },
    error: (err) => {
      this.loading = false;
      if (err.status === 404) {
        this.router.navigate(['/customerdashboard']);
      } else {
        this.errorMessage = 'Could not load your profile. Please try again.';
      }
    }
  });
}
}
}

```

6. components/login/login.component.html : replace the whole file

angularapp/src/app/components/login/login.component.html (replace the whole file)

```

<div class="auth">
  <section class="auth-hero">
    <h1>Learn at your own pace.</h1>
    <p>Browse courses, enroll in a few clicks and share what you thought of them. Pick up where you left off, on any device.</p>
    <ul>
      <li><mat-icon>menu_book</mat-icon>Courses from beginner lessons to certifications</li>
      <li><mat-icon>shopping_cart</mat-icon>Add to cart and enroll in one step</li>
      <li><mat-icon>star</mat-icon>Rate courses to help other learners</li>
    </ul>
  </section>

  <section class="auth-form">
    <div class="form-card">
      <h2>Login</h2>
      <p class="lead">Welcome back. Sign in to continue.</p>
      <form [formGroup]="loginForm" (ngSubmit)="onSubmit()" novalidate>
        <mat-form-field appearance="outline">
          <mat-label>Email</mat-label>
          <input matInput id="email" type="email" formControlName="email" autocomplete="username" />
          <mat-icon matSuffix>mail</mat-icon>
          <mat-error *ngIf="email?.errors?.['required']">Email is required</mat-error>
          <mat-error *ngIf="email?.errors?.['email']">Enter a valid email</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Password</mat-label>
          <input matInput id="password" [type]="showPassword ? 'text' : 'password'"
            formControlName="password" autocomplete="current-password" />
          <button mat-icon-button matSuffix type="button" (click)="showPassword = !showPassword"
            [attr.aria-label]="showPassword ? 'Hide password' : 'Show password'">
            <mat-icon>{{ showPassword ? 'visibility_off' : 'visibility' }}</mat-icon>
          </button>

```

```

    <mat-error *ngIf="password?.errors?.['required']">Password is required</mat-error>
  </mat-form-field>

  <p class="forgot"><a routerLink="/forgot-password" id="forgotLink">Forgot password?</a></p>
  <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{
errorMessage }}<a *ngIf="notRegistered" routerLink="/registration" id="registerFromError" class="err-
link">Register</a></div>
  <button mat-flat-button color="primary" type="submit" id="loginButton"
[disabled]="loginForm.invalid || loading">
    {{ loading ? 'Signing in...' : 'Login' }}
  </button>
</form>

<ng-container *ngIf="googleEnabled">
  <div class="or"><span>or</span></div>
  <div #googleButton id="googleButton" class="google-btn"></div>
</ng-container>

<p class="alt">Don't have an account? <a routerLink="/registration">Register here</a></p>
</div>
</section>
</div>

```

7. login.component.css : add this at the very end

Keep everything that is already in the file. Paste this below it (it uses your Midnight colours):

```

/* Login with Google */
.or { display: flex; align-items: center; gap: 12px; margin: 20px 0 14px; color: var(--muted); font-size:
13px; }
.or::before, .or::after { content: ''; flex: 1; height: 1px; background: var(--line); }
.google-btn { display: flex; justify-content: center; min-height: 44px; }

```

2. Run it and test it

- Step 2 of the backend guide is done; the **same Client ID** is in `application.properties` (or `GOOGLE_CLIENT_ID`) **and** in `environment.ts`.
- Start the backend and `npm start` (port 8081). Open **`http://localhost:8081/login`**. Use the word `localhost`, not `127.0.0.1` (Google checks the exact address you registered).
- You see the form, "or", and the **Continue with Google** button.
- Click it and choose a Gmail you added as a **Test user**. First time: the customer details page. Fill it once; then the dashboard.
- (On the Registration page use **Sign up with Google** for this step; on the Login page an unregistered Gmail must show the "No account found" message and add no row.) Database check: `SELECT user_id, email, username, mobile_number, role FROM users ORDER BY user_id DESC LIMIT 3`; shows your Gmail, your Google name, mobile `0000000000`, role `CUSTOMER`.
- Log out and log in with Google again: the **same** account opens (no second row).
- Email and password login still works; the admin password login still works.
- With OTP on: register a customer with the form, **do not verify**, then use Google with the same Gmail: you land on `/verify-phone`.

Acceptance checklist

- [] Google button visible with a Client ID, hidden without one

- [] Sign up with Google (Registration page) creates a CUSTOMER row and reaches the details page
- [] Sign in with Google (Login page) with a Gmail that never registered shows "No account found ... register first" and adds no row
- [] Sign up with a Gmail that is already registered says "already registered"
- [] Second Google login opens the same account
- [] Form login and admin login unchanged
- [] An unverified form-registered account cannot skip OTP through Google

3. If something goes wrong (frontend and Google)

What you see	Why	Fix
No Google button	<code>googleClientId</code> is <code>''</code> , or the page was not restarted	paste the Client ID, restart <code>npm start</code> , Ctrl+F5
Pop-up says Error 400: origin_mismatch	the address is not in "Authorized JavaScript origins"	add <code>http://localhost:8081</code> and <code>http://localhost</code> , wait 5 minutes, open the site as <code>localhost</code>
Access blocked / app is being tested	that Gmail is not a Test user	Audience, Test users, add it
"Google sign-in failed"	Client ID differs between backend and frontend, or the token expired	use the same Client ID; restart the backend
"Admin accounts must log in..."	that e-mail is an ADMIN account	use the password form
"Login with Google is not set up on the server yet."	backend <code>google.client-id</code> is empty	set it, restart
"Google sign-in could not be loaded"	no internet, or a blocker stops <code>accounts.google.com</code>	check the network, disable the blocker
404 on the customer page after Google login	normal for a new user	fill the customer details form once
If tests fail around the login page		set <code>googleClientId: ''</code>